SaaS Application Discovery & License Right-Sizing

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — SaaS Application Discovery & License Right-Sizing 4
What you get . . . 4
What this delivers . . . 4
The deployment model . . . 4
Outcome . . . 4
1. Solution Overview — SaaS Application Discovery & License Right-Sizing 4
What this solution is . . . 4
How it works . . . 5
The named artifacts (use these names everywhere) . . . 5
What you can control . . . 5
Benefits . . . 6
Limitations (be honest with buyers) . . . 6
The standard deployment model . . . 6
2. Requirements Guide — SaaS Application Discovery & License Right-Sizing 6
2.1 Entra ID sign-in and usage telemetry . . . 6
2.2 Read access to enterprise applications . . . 7
2.3 Microsoft Graph PowerShell access . . . 7
2.4 Sign-in log history sufficient for a usage window . . . 7
2.5 App owner contacts . . . 7
2.6 SaaS admin access where available . . . 7
2.7 A spend baseline . . . 7
Checklist . . . 8
3. Implementation Guide — SaaS Application Discovery & License Right-Sizing 8
Step 1 — Create the review-scope group . . . 8
Step 2 — Discover the SaaS app inventory . . . 8
Step 3 — Read usage and insights . . . 8
Step 4 — Map owners and spend . . . 9
Step 5 — Build the right-sizing register (pilot scope) . . . 9
Step 6 — Flag reclaim candidates safely . . . 9
Step 7 — Validate the pilot (pilot gate) . . . 9
Step 8 — Expand to the full estate . . . 10
Step 9 — Set the recurring review . . . 10
5. Testing Guide — SaaS Application Discovery & License Right-Sizing 10
Test 1 — Inventory is complete . . . 10
Test 2 — Usage telemetry is reading correctly . . . 10

Test 3 — Owner and spend map is filled . . . 10
Test 4 — Right-sizing register identifies real candidates . . . 11
Test 5 — Remediation is safe by default . . . 11
Test 6 — No wrongful reclaim on the pilot . . . 11
Test 7 — Savings realized . . . 11
Pass/fail gate . . . 11
6. Troubleshooting Guide — SaaS Application Discovery & License Right-Sizing 11
A. Discovery returns no apps or no sign-ins . . . 11
B. Connect-MgGraph or scope errors . . . 12
C. Usage window too short . . . 12
D. Suspected wrongful reclaim . . . 12
E. Report seat counts look wrong . . . 12
F. Remediation made a change you did not expect . . . 12
G. When to escalate . . . 13
7. Best Practices — SaaS Application Discovery & License Right-Sizing 13
8. Deliverables Summary — SaaS Application Discovery & License Right-Sizing 14
Documentation . . . 14
PowerShell scripts . . . 14
Named artifacts produced . . . 14
What the buyer receives . . . 15
Outcome . . . 15
Appendix A — PowerShell Scripts 16

Executive Summary — SaaS Application Discovery & License Right-Sizing

SaaS sprawl is the new shadow IT: teams buy apps directly, seats are bought and forgotten, and nobody holds a full picture of SaaS spend. This solution is the repeatable way to discover the SaaS applications your people actually use — from Microsoft Entra ID sign-in and usage telemetry — map each app to an owner and a cost, and right-size licenses by reclaiming unused seats, consolidating overlapping tools, and downgrading over-provisioned plans.

This is a complete, deploy-it-today package for bringing SaaS spend under control, written for a SAM analyst with no prior SaaS-governance experience.

What you get

- A plain-language Solution Overview of SaaS sprawl, discovery via Entra ID sign-ins, the right-sizing levers (reclaim, consolidate, downgrade), and the recurring review.
- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered steps in the Entra admin center and Microsoft Graph.
- Five PowerShell scripts for SaaS app/usage discovery, environment validation, a spend and right-sizing report, safe reclaim-candidate flagging, and a review-scope group builder.
- Testing and Troubleshooting guides for confident rollout.

What this delivers

A named, repeatable workflow: an `App Inventory – SaaS – Discovered`, an `Owner Map – SaaS – Mapped`, a `Right-Sizing Register – SaaS – Pilot` proven on a pilot scope and promoted to the full estate, and a `Review Cadence – SaaS – Recurring` so sprawl does not creep back — with a pilot gate before you scale.

The deployment model

Assess to Configure to Pilot to Validate to Expand.

Outcome

SaaS spend under control — apps discovered, unused and duplicate seats reclaimed, overlapping tools consolidated — with a recurring review to keep it that way, proven on a pilot before you scale.

1. Solution Overview — SaaS Application Discovery & License Right-Sizing

What this solution is

SaaS sprawl is the new shadow IT: teams buy apps directly, seats are bought and forgotten, and nobody holds a full picture of SaaS spend. This solution is the repeatable way to discover the SaaS applications your people actually sign in to, map each app to an owner and a cost, and right-size licenses by reclaiming unused seats, consolidating overlapping tools, and downgrading over-provisioned plans. The discovery signal comes from Microsoft Entra ID — every federated/SSO SaaS app shows up as an enterprise application, and sign-in plus usage telemetry tells you who is really using it.

How it works

1. Entra ID enterprise applications are the inventory of SaaS apps integrated for sign-in. Each one is a candidate for review.
1. Sign-in logs and the Usage and insights reports show which users authenticate to which app, and how recently — the basis for an active-versus-dormant judgement.
1. Each app is mapped to a named owner and an annual cost, producing a spend picture you can rank.
2. Seat utilization (entitled seats versus actively-signing-in users) exposes the three right-sizing levers: reclaim dormant seats, consolidate overlapping tools onto one, and downgrade plans nobody uses the premium features of.
1. A recurring review keeps the picture current as new apps appear and usage shifts, so sprawl does not creep back.

The named artifacts (use these names everywhere)

This solution produces a small set of artifacts. Name each one with the pattern `<Purpose> – <Scope> – <Stage>` and reuse the names verbatim across every document, script, and group:

- `App Inventory – SaaS – Discovered` — the list of SaaS enterprise apps found in Entra ID.
- `Owner Map – SaaS – Mapped` — each app mapped to a business owner and annual cost.
- `Right-Sizing Register – SaaS – Pilot` — the working register of reclaim/consolidate/downgrade candidates for the pilot scope, promoted to `Right-Sizing Register – SaaS – Expanded` for the fleet.
- `Review Cadence – SaaS – Recurring` — the documented schedule and checklist for the ongoing review.
- `Review Scope – SaaS – Pilot` and `Review Scope – SaaS – Expanded` — the Entra ID groups that bound each review pass.

What you can control

- Which apps are in scope (the inventory and the review-scope groups).
- The dormancy threshold (how many days without a sign-in marks a seat as a reclaim candidate).

- The right-sizing action per app: reclaim, consolidate, or downgrade.
- The review cadence (monthly, quarterly) and who owns each app's decision.

Benefits

- Spend control — you see total SaaS spend and the slice that is wasted on dormant or duplicate seats.
- Faster reviews — usage telemetry replaces guesswork and email chains about "who still needs this".
- Less sprawl — a recurring review catches new apps and forgotten seats before they accumulate.
- Auditability — a named inventory, owner map, and register make every decision traceable.

Limitations (be honest with buyers)

- Entra ID sees SaaS apps integrated for sign-in (SSO/federated/gallery). Apps bought outside SSO (personal-card purchases, non-federated tools) are not visible from sign-in logs alone — surface those from expense/procurement data and add them to the inventory manually.
- Sign-in telemetry shows authentication, not feature-level usage. A user who signs in but never uses premium features is a downgrade candidate, which sign-in data alone cannot prove — confirm with the app's own admin console where available.
- Sign-in log retention depends on your Entra ID plan; richer history needs a P1/P2 plan or a Log Analytics export. This is noted in the Requirements Guide.
- Reclaiming a seat is a SaaS-admin action in each vendor's console. This solution finds and flags the candidates; the actual seat removal happens in the app (or via its own automation).

The standard deployment model

> *Discover the inventory, map owners and spend, build the right-sizing register on a pilot scope, validate the reclaim/consolidate/downgrade actions, then expand to the full estate. The repeatable way: Assess to Configure to Pilot to Validate to Expand.*

2. Requirements Guide — SaaS Application Discovery & License Right-Sizing

Verify every requirement before configuring. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites are in `global-requirements-asset-mgt.md`.

2.1 Entra ID sign-in and usage telemetry

Description: Access to Entra ID sign-in logs and the Usage and insights (enterprise apps) reports. Why: Sign-in activity is how you tell an actively-used SaaS seat from a dormant one. Verify: Go to `entra.microsoft.com` then Identity then Applications then Enterprise applications then Usage and insights — the report loads and shows app sign-in counts. Run `SAASLIC-Discovery.ps1`. Implement: Ensure auditing is on (it is by default); for history beyond the default retention, see 2.4.

2.2 Read access to enterprise applications

Description: A role that can read enterprise applications and sign-in logs — at minimum Reports Reader plus Application Reader, or a Global Reader. Why: Discovery and reporting read the app list, assignments, and sign-in events. Verify: At `entra.microsoft.com` then Identity then Applications then Enterprise applications, the list loads and you can open an app's Sign-in logs and Users and groups. Implement: Assign Reports Reader and Application Reader (or Global Reader) to the SAM analyst account.

2.3 Microsoft Graph PowerShell access

Description: The Microsoft Graph PowerShell SDK installed, with consent to read directory, apps, and audit (sign-in) data. Why: The scripts in this solution read the app inventory and sign-in telemetry through Graph. Verify: `Connect-MgGraph -Scopes 'Application.Read.All','AuditLog.Read.All','Directory.Read.All'` succeeds. Run `SAASLIC-Validation.ps1`. Implement: `Install-Module Microsoft.Graph -Scope CurrentUser`; grant the read scopes above when prompted.

2.4 Sign-in log history sufficient for a usage window

Description: Sign-in data covering at least your chosen dormancy window (e.g. 30, 60, or 90 days). Why: A seat is only a reliable reclaim candidate if it has been dormant across the full window. Verify: In Sign-in logs, the date filter reaches back to your window; or query via `SAASLIC-Discovery.ps1` and confirm the earliest event predates the window. Implement: Entra ID retains sign-in logs for a limited period by default (longer with P1/P2). For a longer window, export sign-in logs to a Log Analytics workspace and query there.

2.5 App owner contacts

Description: A named business owner for each in-scope SaaS app. Why: Right-sizing decisions (reclaim, consolidate, downgrade) need an accountable owner to approve them. Verify: For a sample app, you can name the owner; record it in `Owner Map – SaaS – Mapped`. Implement: Identify owners from procurement records, the app's admin, or the team that requested it; fill any gaps during the pilot.

2.6 SaaS admin access where available

Description: Admin (or read) access to each in-scope app's own admin console. Why: Sign-in telemetry flags candidates; the actual seat reclaim/downgrade happens in the app's console, and feature-level usage (for downgrades) is only visible there. Verify: For a representative app, its admin console loads and shows the seat/license list. Implement: Request least-privilege admin/read access per app from each app owner.

2.7 A spend baseline

Description: Current annual cost (and seat count purchased) per in-scope SaaS app. Why: Spend is what ranks the review and quantifies the savings from right-sizing. Verify: For a sample app, you can state its annual cost and entitled seats from contracts/invoices. Implement: Pull costs from procurement/finance; record them in `Owner Map – SaaS – Mapped`.

Checklist

- Entra ID sign-in + Usage and insights reports accessible
- Read access to enterprise applications and sign-in logs
- Microsoft Graph PowerShell installed with read scopes — `SAASLIC-Validation.ps1`
- Sign-in history covers the dormancy window
- App owners identified — `Owner Map – SaaS – Mapped`
- SaaS admin/read access where available
- Spend baseline recorded per app
- `Review Scope – SaaS – Pilot` group created

3. Implementation Guide — SaaS Application Discovery & License Right-Sizing

Follow these steps in order. Discovery and reporting use the Microsoft Entra admin center (`entra.microsoft.com`) and the Microsoft Graph PowerShell scripts; right-sizing actions happen in each SaaS app's own admin console.

> *Model: discover the inventory, map owners and spend, build a right-sizing register on a pilot scope, validate the actions, then expand. A pilot gate sits between Validate and Expand.*

Step 1 — Create the review-scope group

1. Go to `entra.microsoft.com` then Identity then Groups then All groups then New group.
2. Group type Security; Name `Review Scope – SaaS – Pilot`. Add a representative set of users (one

department or team whose SaaS use you understand). Create.

1. You can build or refresh this group later with `SAASLIC-Set-ScopeTargeting.ps1`.

Step 2 — Discover the SaaS app inventory

1. Go to `entra.microsoft.com` then Identity then Applications then Enterprise applications. This list

is your raw SaaS inventory (apps integrated for sign-in).

1. Run `SAASLIC-Discovery.ps1` to export the enterprise apps plus per-app sign-in activity to a CSV.
2. Record the result as `App Inventory – SaaS – Discovered`. Add any apps bought outside SSO (from

expense/procurement data) manually — sign-in logs cannot see those.

Step 3 — Read usage and insights

1. Go to `entra.microsoft.com` then Identity then Applications then Enterprise applications then Usage

and insights. Review sign-in counts per app over the last 7/30 days.

1. Cross-check against `SAASLIC-Discovery.ps1` output. Flag apps with few or zero sign-ins as

consolidation/retirement candidates.

Step 4 — Map owners and spend

1. For each app in `App Inventory – SaaS – Discovered`, add the business owner (Requirement 2.5) and

the annual cost plus entitled seats (Requirement 2.7).

1. Save this as `Owner Map – SaaS – Mapped`. Sort by annual cost so the highest-spend apps are reviewed first.

Step 5 — Build the right-sizing register (pilot scope)

1. Run `SAASLIC-Report.ps1` to produce, per app, the entitled seats versus actively-signing-in users

over your dormancy window, with a wasted-seat and estimated-savings figure.

1. For each app, choose a right-sizing lever and record it in `Right-Sizing Register – SaaS – Pilot`:

• Reclaim — users with no sign-in across the window; remove their seat.

• Consolidate — two or more apps doing the same job; pick one, migrate users off the others.

• Downgrade — users who sign in but do not need the premium plan; move them to a cheaper tier.

1. Get each app owner's approval against the register before any seat is changed.

Step 6 — Flag reclaim candidates safely

1. Run `SAASLIC-Remediation.ps1` (default `-WhatIf`). It lists the dormant-seat reclaim candidates and

writes a candidate file — it changes nothing by default.

1. Review the candidate list with the owner. The actual seat removal/downgrade is performed in each

SaaS app's own admin console (or its automation); this solution flags, it does not delete vendor seats.

Step 7 — Validate the pilot (pilot gate)

1. Confirm reclaimed users in the pilot were genuinely dormant and have not lost needed access — check

for any access request or complaint after the change.

1. Re-run `SAASLIC-Report.ps1` and confirm the seat counts and spend for the pilot apps dropped as

expected. Complete the Testing Guide.

1. Pilot gate: only proceed to Step 8 when the pilot apps pass the Testing Guide and no wrongful reclaim occurred.

Step 8 — Expand to the full estate

1. Build `Review Scope – SaaS – Expanded` (the rest of the organization) with `SAASLIC-Set-ScopeTargeting.ps1`.

1. Repeat Steps 2 to 6 across the full inventory; promote the register to `Right-Sizing Register – SaaS – Expanded`.

Step 9 — Set the recurring review

1. Document the cadence (e.g. monthly for high-spend apps, quarterly for the rest) as `Review Cadence – SaaS – Recurring`: who runs the scripts, who approves, and the dormancy threshold.

1. Schedule it; each cycle re-runs Discovery and Report, refreshes the register, and reclaims newly dormant seats — so sprawl does not creep back.

Order of operations

```
Scope group -> Discover inventory -> Usage and insights -> Owner & spend map ->
Right-sizing register -> Flag reclaim (WhatIf) -> Validate (pilot gate) ->
Expand -> Recurring review
```

5. Testing Guide — SaaS Application Discovery & License Right-Sizing

Run after the pilot register is built and reclaim candidates are flagged (Implementation Steps 5 to 7), and after each recurring review cycle.

Test 1 — Inventory is complete

Do: Compare `App Inventory – SaaS – Discovered` (from `SAASLIC-Discovery.ps1`) against the Enterprise applications list at `entra.microsoft.com` then Identity then Applications then Enterprise applications. Expected: Every SSO-integrated SaaS app appears; known non-SSO apps were added manually.

Test 2 — Usage telemetry is reading correctly

Do: For a sample app, compare the script's sign-in count to Usage and insights for the same window. Expected: The counts agree (within the report's reporting lag).

Test 3 — Owner and spend map is filled

Do: Open `Owner Map – SaaS – Mapped`. Expected: Every in-scope app has a named owner and an annual cost; the map sorts by spend.

Test 4 — Right-sizing register identifies real candidates

Do: Run `SAASLIC-Report.ps1` for the pilot scope. Expected: Each app shows entitled seats, active users, wasted seats, and an estimated saving; the register's reclaim/consolidate/downgrade choices follow the data.

Test 5 — Remediation is safe by default

Do: Run `SAASLIC-Remediation.ps1` with no switches. Expected: It runs in `-WhatIf` mode, lists reclaim candidates, writes the candidate file, and changes nothing.

Test 6 — No wrongful reclaim on the pilot

Do: After the pilot reclaim actions, check for access requests or complaints from reclaimed users. Expected: No genuinely-active user lost a needed seat.

Test 7 — Savings realized

Do: Re-run `SAASLIC-Report.ps1` after the pilot reclaim and compare to the baseline. Expected: Entitled-seat counts and estimated spend for the pilot apps dropped as planned.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1 to 6 pass; no wrongful reclaim on the pilot scope
Expand pass	Tests 1, 4, 7 pass for the expanded scope; savings confirmed against baseline

6. Troubleshooting Guide — SaaS Application Discovery & License Right-Sizing

Use `SAASLIC-Validation.ps1` for connectivity/scopes, `SAASLIC-Discovery.ps1` for the inventory and sign-in activity, `SAASLIC-Report.ps1` for seat utilization, and `SAASLIC-Remediation.ps1` (default `-WhatIf`) to flag reclaim candidates.

A. Discovery returns no apps or no sign-ins

Symptom	Cause	Remediation
Empty app list	Read access missing	Confirm Reports Reader + Application Reader (or Global Reader); re-run `SAASLIC-Validation.ps1`
Apps listed, zero sign-ins	Audit/sign-in scope not consented, or no recent activity	Re-consent `AuditLog.Read.All`; widen the window; confirm the app is SSO-integrated
Known app is missing	App bought outside SSO (not federated)	Add it to `App Inventory – SaaS – Discovered` manually from expense/procurement data

B. Connect-MgGraph or scope errors

- `SAASLIC-Validation.ps1` reports a missing scope: reconnect with

`Connect-MgGraph -Scopes 'Application.Read.All','AuditLog.Read.All','Directory.Read.All'` and consent.

- Module not found: `Install-Module Microsoft.Graph -Scope CurrentUser -Force`.

C. Usage window too short

Symptom	Cause	Remediation
Few events / window incomplete	Sign-in log retention shorter than the dormancy window	Use a shorter window, or export sign-in logs to a Log Analytics workspace and query there (Requirement 2.4)

D. Suspected wrongful reclaim

- A reclaimed user complains: confirm the user's sign-in history in `SAASLIC-Report.ps1` for the window;

if they did sign in, the dormancy threshold was too tight — lengthen it and restore the seat in the app's console.

E. Report seat counts look wrong

- Entitled seats come from your spend baseline (`Owner Map – SaaS – Mapped`), not from Entra ID — verify

the entitled-seat figure against the vendor invoice/console. Sign-in-derived active users only count authentications, not feature use; confirm downgrade candidates in the app's admin console.

F. Remediation made a change you did not expect

• SAASLIC-Remediation.ps1 is read-only/flag-only by default (-WhatIf); it writes a candidate file

but does not touch vendor seats. Actual seat removal happens in each SaaS app's own console — review the candidate file with the owner before acting there.

G. When to escalate

If discovery, validation, and reporting all run clean, the scopes are consented, and the inventory still omits an app you can see in the portal, capture the SAASLIC-Discovery.ps1 output and the app's properties from Enterprise applications and open a Microsoft support case.

7. Best Practices — SaaS Application Discovery & License Right-Sizing

• Pilot first, always. Right-size one well-understood scope before the fleet — it proves the dormancy

threshold is right and catches wrongful reclaims before they reach real users.

• Default to WhatIf. Never let a reclaim run unattended; review the candidate list with the owner, then

perform the seat change in the app's own console.

• Map every app to an owner. An app with no owner is an app nobody will approve decisions for — fill the

Owner Map – SaaS – Mapped gaps before reclaiming anything.

• Rank by spend. Review the highest-cost apps first; that is where right-sizing returns the most.

• Pick a defensible dormancy window. 30 days catches the most waste but risks seasonal users; 60 to 90

days is safer for tools used intermittently. Document the choice.

• Confirm downgrades in the app. Sign-in data proves authentication, not feature use — verify a

downgrade candidate in the vendor's admin console before moving their plan.

• Catch non-SSO sprawl. Sign-in logs miss apps bought outside SSO; reconcile against expense/procurement

data so the inventory is complete.

• Consolidate overlaps deliberately. When two apps do the same job, migrate users to one and retire the

other — do not just trim seats on both.

• Make it recurring. A one-time clean-up decays; Review Cadence – SaaS – Recurring keeps sprawl from

creeping back.

• Keep it traceable. Name every artifact with <Purpose> – <Scope> – <Stage> and reuse the names, so

any decision can be traced from inventory to register to action for audit.

8. Deliverables Summary — SaaS Application Discovery & License Right-Sizing

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	SaaS sprawl, discovery via Entra ID sign-ins/telemetry, right-sizing levers, the named artifacts, benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Asset Mgt)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: discover, map owners/spend, build the register, flag reclaims, validate, expand, recur
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`SAASLIC-Discovery.ps1`	Read-only. Lists Entra ID enterprise (SaaS) apps with per-app sign-in activity (Microsoft Graph)
`SAASLIC-Validation.ps1`	Read-only. Confirms Graph connectivity and the read scopes/telemetry needed to run the solution
`SAASLIC-Report.ps1`	Read-only. Per-app entitled-seats vs active-users, wasted seats, and estimated savings (Microsoft Graph)
`SAASLIC-Remediation.ps1`	Default `-WhatIf`. Flags dormant-seat reclaim candidates; writes a candidate file, changes nothing by default
`SAASLIC-Set-ScopeTargeting.ps1`	Builds/refreshes the `Review Scope – SaaS` Entra ID group that bounds each review pass

Named artifacts produced

- `App Inventory – SaaS – Discovered`
- `Owner Map – SaaS – Mapped`

- `Right-Sizing Register – SaaS – Pilot` / `Right-Sizing Register – SaaS – Expanded`
- `Review Cadence – SaaS – Recurring`
- `Review Scope – SaaS – Pilot` / `Review Scope – SaaS – Expanded`

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: Assess to Configure to Pilot to Validate to Expand.

Outcome

SaaS spend under control — apps discovered, unused and duplicate seats reclaimed, overlapping tools consolidated — with a recurring review to keep it that way, proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SAASLIC-Discovery.ps1

Purpose: Builds the 'App Inventory - SaaS - Discovered' artifact: every enterprise application in Entra ID (the SSO-integrated SaaS estate) with its app sign-in count and last sign-in over a window. Read-only — it reads the directory and the sign-in audit logs, and changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SAASLIC-Discovery.ps1 -Days 30 -CsvPath C:\Temp\saas-inventory.csv Requires the Microsoft Graph PowerShell SDK and the scopes Application.Read.All and AuditLog.Read.All (and Directory.Read.All). See SAASLIC-Validation.ps1. EXPECTED RESULTS A table of SaaS apps with sign-in counts and last sign-in, and (with -CsvPath) a CSV export to record as 'App Inventory - SaaS - Discovered'. Add non-SSO apps manually afterwards. .NOTES Workplace Marketplace — Asset Mgt. Read-only against Microsoft Graph. #> [CmdletBinding()] param([int]$Days = 30, [string]$CsvPath) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Application.Read.All','AuditLog.Read.All','Directory.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\SAASLIC-Discovery.ps1 -Days 30 -CsvPath C:\Temp\saas-inventory.csv Requires the Microsoft Graph PowerShell SDK and the scopes Application.Read.All and AuditLog.Read.All (and Directory.Read.All). See SAASLIC-Validation.ps1. EXPECTED RESULTS A table of SaaS apps with sign-in counts and last sign-in, and (with -CsvPath) a CSV export to record as 'App Inventory - SaaS - Discovered'. Add non-SSO apps manually afterwards. .NOTES Workplace Marketplace — Asset Mgt. Read-only against Microsoft Graph. #> [CmdletBinding()] param([int]$Days = 30, [string]$CsvPath) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Application.Read.All','AuditLog.Read.All','Directory.Read.All' -NoWelcome

SAASLIC-Remediation.ps1

Purpose: For a given app, lists the users who were assigned the app in Entra ID but have NOT signed in across the dormancy window — the reclaim candidates. By default it runs in -WhatIf mode: it only reports and writes a candidate file and changes nothing. The actual seat removal happens in the SaaS app's own admin console after the owner approves; this script does not delete vendor seats or Entra assignments unless you explicitly opt out of -WhatIf, in which case it only removes the Entra ID app *assignment* for confirmed candidates (still not the vendor seat). HOW TO RUN Flag only (default, safe): powershell -ExecutionPolicy Bypass -File .\SAASLIC-Remediation.ps1 -AppId -Days 30 -CsvPath C:\Temp\reclaim.csv Remove the Entra ID app assignment for candidates (requires AppRoleAssignment.ReadWrite.All): powershell -ExecutionPolicy Bypass -File .\SAASLIC-Remediation.ps1 -AppId -Days 30 -WhatIf:$false EXPECTED RESULTS A list of dormant assigned users (reclaim candidates) and a candidate CSV. With -WhatIf (default) nothing is changed — review the file with the app owner before acting in the app. .NOTES Workplace Marketplace — Asset Mgt. Read-only by default; -WhatIf is ON unless disabled. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)][string]$AppId, [int]$Days = 30, [string]$CsvPath) # Default to safe (WhatIf) unless the caller explicitly passed -WhatIf:$false if (-not

$PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = $true }

How to run: Flag only (default, safe): powershell -ExecutionPolicy Bypass -File .\SAASLIC-Remediation.ps1 -AppId <appId> -Days 30 -CsvPath C:\Temp\reclaim.csv Remove the Entra ID app assignment for candidates (requires AppRoleAssignment.ReadWrite.All): powershell -ExecutionPolicy Bypass -File .\SAASLIC-Remediation.ps1 -AppId <appId> -Days 30 -WhatIf:$false EXPECTED RESULTS A list of dormant assigned users (reclaim candidates) and a candidate CSV. With -WhatIf (default) nothing is changed — review the file with the app owner before acting in the app. .NOTES Workplace Marketplace — Asset Mgt. Read-only by default; -WhatIf is ON unless disabled. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)][string]$AppId, [int]$Days = 30, [string]$CsvPath) # Default to safe (WhatIf) unless the caller explicitly passed -WhatIf:$false if (-not $PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = $true }

SAASLIC-Report.ps1

Purpose: Produces the data behind 'Right-Sizing Register - SaaS - Pilot': per app, the entitled seats and annual cost (from your spend baseline), the count of distinct users who actually signed in over the window, the wasted (idle) seats, and the estimated annual saving from reclaiming them. Read-only — it reads sign-in telemetry from Graph and your owner-map CSV; it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SAASLIC-Report.ps1 -OwnerMapCsv .\owner-map.csv -Days 30 -CsvPath C:\Temp\rightsizing.csv OwnerMapCsv columns expected: AppId, App, Owner, EntitledSeats, AnnualCost (AnnualCost is the total annual cost for the app; per-seat cost is derived). Requires Microsoft Graph SDK and scopes AuditLog.Read.All, Directory.Read.All. EXPECTED RESULTS A per-app table (entitled / active / wasted seats / estimated saving) to record as the 'Right-Sizing Register - SaaS - Pilot'. Use it to choose reclaim / consolidate / downgrade. .NOTES Workplace Marketplace — Asset Mgt. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$OwnerMapCsv, [int]$Days = 30, [string]$CsvPath) if (-not (Test-Path $OwnerMapCsv)) { throw "OwnerMapCsv not found: $OwnerMapCsv (build 'Owner Map - SaaS - Mapped' first)." } if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'AuditLog.Read.All','Directory.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\SAASLIC-Report.ps1 -OwnerMapCsv .\owner-map.csv -Days 30 -CsvPath C:\Temp\rightsizing.csv OwnerMapCsv columns expected: AppId, App, Owner, EntitledSeats, AnnualCost (AnnualCost is the total annual cost for the app; per-seat cost is derived). Requires Microsoft Graph SDK and scopes AuditLog.Read.All, Directory.Read.All. EXPECTED RESULTS A per-app table (entitled / active / wasted seats / estimated saving) to record as the 'Right-Sizing Register - SaaS - Pilot'. Use it to choose reclaim / consolidate / downgrade. .NOTES Workplace Marketplace — Asset Mgt. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$OwnerMapCsv, [int]$Days = 30, [string]$CsvPath) if (-not (Test-Path $OwnerMapCsv)) { throw "OwnerMapCsv not found: $OwnerMapCsv (build 'Owner Map - SaaS - Mapped' first)." } if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'AuditLog.Read.All','Directory.Read.All' -NoWelcome

SAASLIC-Set-ScopeTargeting.ps1

Purpose: Creates (if missing) and populates the security group that bounds a review pass — e.g. 'Review Scope - SaaS - Pilot' for the pilot or 'Review Scope - SaaS - Expanded' for the fleet. Members are added from a CSV of user principal names so each recurring review targets a known, repeatable set of users. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SAASLIC-Set-ScopeTargeting.ps1 -GroupName "Review Scope - SaaS - Pilot" -MembersCsv .\pilot-users.csv MembersCsv expects a single column: UserPrincipalName Requires Microsoft Graph SDK and scopes Group.ReadWrite.All, User.Read.All. EXPECTED RESULTS The named group exists and contains the listed users; new members are added, existing ones left in place (idempotent). Use the group as 'Review Scope - SaaS - Pilot/Expanded'. .NOTES Workplace Marketplace — Asset Mgt. Writes group membership; safe to re-run (idempotent). #> [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)][string]$GroupName, [Parameter(Mandatory)][string]$MembersCsv) if (-not (Test-Path $MembersCsv)) { throw "MembersCsv not found: $MembersCsv (column: UserPrincipalName)." } if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All','User.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\SAASLIC-Set-ScopeTargeting.ps1 -GroupName "Review Scope - SaaS - Pilot" -MembersCsv .\pilot-users.csv MembersCsv expects a single column: UserPrincipalName Requires Microsoft Graph SDK and scopes Group.ReadWrite.All, User.Read.All. EXPECTED RESULTS The named group exists and contains the listed users; new members are added, existing ones left in place (idempotent). Use the group as 'Review Scope - SaaS - Pilot/Expanded'. .NOTES Workplace Marketplace — Asset Mgt. Writes group membership; safe to re-run (idempotent). #> [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)][string]$GroupName, [Parameter(Mandatory)][string]$MembersCsv) if (-not (Test-Path $MembersCsv)) { throw "MembersCsv not found: $MembersCsv (column: UserPrincipalName)." } if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All','User.Read.All' -NoWelcome

SAASLIC-Validation.ps1

Purpose: Verifies the Microsoft Graph PowerShell SDK is present, that you can connect, that the required read scopes are granted (Application.Read.All, AuditLog.Read.All, Directory.Read.All), and that enterprise apps and sign-in logs are readable. Read-only — it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SAASLIC-Validation.ps1 EXPECTED RESULTS "ENVIRONMENT READY" when the SDK, connection, scopes, and sample reads all succeed. .NOTES Workplace Marketplace — Asset Mgt. Read-only against Microsoft Graph. #> [CmdletBinding()] param() Write-Host "=== SAASLIC environment validation ===" -ForegroundColor Cyan $need = 'Application.Read.All','AuditLog.Read.All','Directory.Read.All' $ok = $true

How to run: powershell -ExecutionPolicy Bypass -File .\SAASLIC-Validation.ps1 EXPECTED RESULTS "ENVIRONMENT READY" when the SDK, connection, scopes, and sample reads all succeed. .NOTES Workplace Marketplace — Asset Mgt. Read-only against Microsoft Graph. #> [CmdletBinding()] param() Write-Host "=== SAASLIC environment validation ===" -ForegroundColor Cyan $need = 'Application.Read.All','AuditLog.Read.All','Directory.Read.All' $ok = $true